



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Martin Andrés Morales Mazariegos **Carné:** 1217025

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?
Una API REST es un estilo de arquitectura de software para interconectar aplicaciones a través del protocolo HTTP. Utiliza los métodos estándar de la web para realizar operaciones sobre recursos de manera ligera, escalable y sin guardar estado de la sesión.
2. ¿Qué es un endpoint?
Un endpoint es la dirección específica (URI) donde una API expone un recurso determinado para que los clientes interactúen con él. Representa el punto exacto al que se envía una solicitud HTTP para realizar una acción.
3. ¿Qué diferencia existe entre una URL y un endpoint?
URL (Uniform Resource Locator): Es la dirección web completa utilizada para ubicar cualquier recurso en la red.
Endpoint: Es la combinación de la URL base con una ruta específica y un método HTTP que representa una funcionalidad o recurso concreto.
4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?
 - GET: Solicita o recupera datos de un recurso existente sin modificarlo.
 - POST: Envía información al servidor para crear un nuevo recurso.PUT / PATCH:
 - PUT: Reemplaza completamente un recurso existente con los nuevos datos enviados.
 - PATCH: Modifica o actualiza parcialmente un recurso (solo los campos enviados).
 - DELETE: Elimina un recurso específico del servidor.
5. ¿Qué es JSON y por qué es común en las APIs?
JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos estructurado en pares de clave-valor ({"nombre": "Ana", "edad": 30}). Es el formato dominante en las APIs REST porque resulta fácilmente legible tanto para humanos como para máquinas, es agnóstico al lenguaje de programación, contando con soporte nativo en todos los lenguajes modernos y requiere menos ancho de banda que alternativas más pesadas como XML.
6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.
 - 200 OK: La solicitud fue exitosa y el servidor devuelve la respuesta esperada.
 - 404 Not Found: El recurso solicitado o el endpoint no existe en el servidor (error del cliente).
 - 500 Internal Server Error: El servidor encontró un problema inesperado o un fallo en el código al procesar la solicitud (error del servidor).



7. ¿Qué información se puede encontrar en la respuesta de una API?
Una respuesta típica de una API incluye un código de estado HTTP que indica el éxito o fallo de la petición, encabezados con metadatos sobre el servidor y la seguridad, y un cuerpo que contiene los datos solicitados en formato JSON o los detalles de un error.
8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?
Probar una API con Postman antes de escribir código permite validar el diseño mediante respuestas simuladas, facilita acordar el contrato de datos con el equipo de frontend o consumidores externos y ahorra tiempo al detectar problemas de arquitectura de manera temprana.
9. ¿Qué es y para qué sirve Swagger UI?
Swagger UI es una herramienta que genera documentación visual e interactiva a partir de una especificación OpenAPI. Sirve para que desarrolladores y clientes puedan visualizar los endpoints disponibles, ver los parámetros y estructuras esperadas, e incluso probar las peticiones directamente desde el navegador web sin instalar software adicional.
10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?
Postman actúa como un cliente HTTP completo y entorno de pruebas que permite construir y enviar solicitudes personalizadas, inspeccionar respuestas detalladas como latencias y códigos de estado, organizar peticiones en colecciones reusables y automatizar pruebas unitarias y de integración sobre los endpoints.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
- Indique el método HTTP utilizado.
- Indique qué parámetro necesita la solicitud.
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
- Explique con sus palabras qué información contiene la respuesta.

Evidencia: captura de Swagger mostrando la operación y su respuesta.



2.1.

The screenshot shows the Swagger UI for the petstore3 API. The endpoint `GET /pet/{petId}` is selected, with the description "Find pet by ID." and "Returns a single pet." The parameter `petId` is defined as a required integer (int64) in the path. The response is a 200 OK status with a successful operation, returning a JSON object representing a pet. The example JSON is:

```
{
  "id": 10,
  "name": "doggie",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    "tag"
  ]
}
```

On the right, the Chrome DevTools Network tab shows the request details for `openapi.json`. The request is a GET to `https://petstore3.swagger.io/api/v3/openapi.json` with a status of 200 OK. The response headers include `Content-Type: application/json` and `Date: Wed, 30 Sep 2026 13:36:39 GMT`.

2.2. GET

2.3. getId

3. Crear un recurso con POST

Utilice POST `/pet` para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET `/pet/{petId}`.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT `/pet`. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
- ¿Qué datos fueron enviados en el body?
- ¿Qué código HTTP devolvió la API?
- ¿Cómo comprobó que el cambio realmente ocurrió?



5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
- Explique dónde se envía ese parámetro.
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:



- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.